

cms_doubleelectron_zcycle_v1

June 15, 2026

1 OTUS | CMS DoubleElectron with MG5 Z/γ^* Prior

This notebook trains OTUS using CMS Open Data DoubleElectron events as detector-level data x , and a standalone MadGraph5 pre-detector Drell–Yan $pp \rightarrow e^+ e^-$ prior as latent/theory-level data z .

Expected z HDF5 file layout:

```
FDL/zData : shape (N, 8)
columns   : [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]
```

Put the MG5 HDF5 file in the same folder as this notebook, or set `theory_prior_file` to its full path in the meta-parameter cell.

2 Load Required Libraries

```
[33]: %matplotlib inline
from pathlib import Path

import awkward as ak
import matplotlib.pyplot as plt
import numpy as np
import torch
import uproot

import sys
from pathlib import Path

import os

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

# Change this to your actual OTUS root folder
OTUS_ROOT = Path(r"C:\Users\AhrixMarin\Desktop\otus")
UTILITY_DIR = OTUS_ROOT / "utilityFunctions"
```

```

assert (UTILITY_DIR / "func_utils.py").exists(), f"Cannot find func_utils.py in {UTILITY_DIR}"

sys.path.insert(0, str(UTILITY_DIR))

print("Using utilityFunctions from:", UTILITY_DIR)

## Determine if using GPU or CPU ##
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0' # Set to '-1' to disable GPU

from configs import device

print('Using device:', device)

```

Using utilityFunctions from: C:\Users\AhrixMarin\Desktop\otus\utilityFunctions
Using device: cuda

3 Meta Parameters

```

[34]: cms_root_file = Path('../cms_zpeak/data/Run2012B_DoubleElectron.root')

# MG5 prior produced on Linux and copied to this notebook folder.
# Change this filename if your HDF5 file has a different name.
theory_prior_filename = 'cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5'
theory_prior_file = Path(theory_prior_filename)

# Used only for saved model/results filenames.
dataset_name = 'cms_doubleelectron_mg5_dyee_dy1j_ptj5_zcycle_v1'

## Set random seeds ##
seed = 0
torch.manual_seed(seed)
np.random.seed(seed)

## Set data type ##
from configs import float_type
print('Using data type:', float_type)
print('Notebook/kernel working directory:', Path.cwd())
print('Expected MG5 prior file:', theory_prior_file)
print('Local .hdf5 files:', sorted([p.name for p in Path('.').glob('*.hdf5')]))

```

Using data type: float32
Notebook/kernel working directory:
c:\Users\AhrixMarin\Desktop\otus\experiments\cms_doubleelectron
Expected MG5 prior file: cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5
Local .hdf5 files: ['cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5',

```
'cms_dyee_mg5_8tev_fiducial_70_110.hdf5']
```

4 Load CMS DoubleElectron ROOT File

```
[35]: assert cms_root_file.exists(), f'File not found: {cms_root_file.resolve()}'
```

```
cms_file = uproot.open(cms_root_file)
events = cms_file['Events']

print('File:', cms_root_file.resolve())
print('Number of events:', events.num_entries)

electron_branches = [k for k in events.keys() if 'Electron' in k]
electron_branches[:30]
```

File: C:\Users\AhrixMarin\Desktop\otus\experiments\cms_zpeak\data\Run2012B_DoubleElectron.root

Number of events: 21474287

```
[35]: ['nElectron',
      'Electron_pt',
      'Electron_eta',
      'Electron_phi',
      'Electron_mass',
      'Electron_charge',
      'Electron_pfRelIso03_all',
      'Electron_dxy',
      'Electron_dxyErr',
      'Electron_dz',
      'Electron_dzErr']
```

```
[36]: branches = [
      'nElectron',
      'Electron_pt',
      'Electron_eta',
      'Electron_phi',
      'Electron_mass',
      'Electron_charge',
      'Electron_pfRelIso03_all',
      'Electron_dxy',
      'Electron_dz',
  ]

arrays = events.arrays(branches, library='ak')

print('Loaded events:', len(arrays['nElectron']))
print('Example nElectron values:', arrays['nElectron'][:10])
```

Loaded events: 21474287

Example nElectron values: [0, 1, 2, 2, 1, 2, 0, 2, 1, 0]

5 Select Good Electron Pairs

```
[37]: # Basic kinematic cuts
electron_pt_min = 20.0
electron_abs_eta_max = 2.5

# Electron quality cuts
electron_iso_max = 0.15
electron_dxy_max = 0.05
electron_dz_max = 0.10

# Z-window cut
z_mass_min = 70.0
z_mass_max = 110.0

electrons = ak.zip({
    'pt': arrays['Electron_pt'],
    'eta': arrays['Electron_eta'],
    'phi': arrays['Electron_phi'],
    'mass': arrays['Electron_mass'],
    'charge': arrays['Electron_charge'],
    'pfRelIso03_all': arrays['Electron_pfRelIso03_all'],
    'dxy': arrays['Electron_dxy'],
    'dz': arrays['Electron_dz'],
})

abs_eta = np.abs(electrons.eta)
outside_ecal_gap = ~((abs_eta > 1.4442) & (abs_eta < 1.566))

selected_electrons = electrons[
    (electrons.pt > electron_pt_min)
    & (abs_eta < electron_abs_eta_max)
    & outside_ecal_gap
    & (electrons.pfRelIso03_all < electron_iso_max)
    & (np.abs(electrons.dxy) < electron_dxy_max)
    & (np.abs(electrons.dz) < electron_dz_max)
]

print('Total selected electrons:', ak.sum(ak.num(selected_electrons)))
print('Events with >=2 selected electrons:', ak.sum(ak.num(selected_electrons)
    ↪ >= 2))

pairs = ak.combinations(selected_electrons, 2, fields=['e1', 'e2'])
os_pairs = pairs[(pairs.e1.charge * pairs.e2.charge) < 0]
```

```

e_minus = ak.where(os_pairs.e1.charge < 0, os_pairs.e1, os_pairs.e2)
e_plus = ak.where(os_pairs.e1.charge > 0, os_pairs.e1, os_pairs.e2)

def pair_mass(first, second):
    px = first.pt * np.cos(first.phi) + second.pt * np.cos(second.phi)
    py = first.pt * np.sin(first.phi) + second.pt * np.sin(second.phi)
    pz = first.pt * np.sinh(first.eta) + second.pt * np.sinh(second.eta)
    e1 = np.sqrt((first.pt * np.cosh(first.eta)) ** 2 + first.mass ** 2)
    e2 = np.sqrt((second.pt * np.cosh(second.eta)) ** 2 + second.mass ** 2)
    mass2 = (e1 + e2) ** 2 - px ** 2 - py ** 2 - pz ** 2
    return np.sqrt(ak.where(mass2 > 0, mass2, 0))

m_ee = pair_mass(e_minus, e_plus)
z_window = (m_ee > z_mass_min) & (m_ee < z_mass_max)

e_minus = ak.flatten(e_minus[z_window])
e_plus = ak.flatten(e_plus[z_window])
masses_ee = ak.to_numpy(ak.flatten(m_ee[z_window]))

print('Opposite-sign dielectron pairs in 70-110 GeV:', len(masses_ee))
print(f'Mean mass: {np.mean(masses_ee):.6f} GeV')
print(f'Median mass: {np.median(masses_ee):.6f} GeV')

```

```

Total selected electrons: 9734445
Events with >=2 selected electrons: 1903393
Opposite-sign dielectron pairs in 70-110 GeV: 1344681
Mean mass: 90.638089 GeV
Median mass: 91.148434 GeV

```

6 Build Detector-Level x-Space Array

```

[38]: def p4_array(particle):
    pt = ak.to_numpy(particle.pt)
    eta = ak.to_numpy(particle.eta)
    phi = ak.to_numpy(particle.phi)
    mass = ak.to_numpy(particle.mass)

    px = pt * np.cos(phi)
    py = pt * np.sin(phi)
    pz = pt * np.sinh(eta)
    energy = np.sqrt(px ** 2 + py ** 2 + pz ** 2 + mass ** 2)
    return np.stack([px, py, pz, energy], axis=1)

# Match ppzee ordering: [p_e-^mu, p_e+^mu], with p^mu = [px, py, pz, E]
x_data = np.concatenate([p4_array(e_minus), p4_array(e_plus)], axis=1)

```

```
print('x_data shape:', x_data.shape)
print('First x row:', x_data[0])
```

```
x_data shape: (1344681, 8)
First x row: [-35.98058   48.777832  35.590557  70.28917   20.748224 -27.40425
 26.977688  43.69528 ]
```

7 Load Theory-Level z-Space Prior

```
[39]: import h5py

# Load the MadGraph5 pre-detector physics prior z from the HDF5 file.
# Expected HDF5 layout:
#   FDL/zData : shape (N, 8)
# with columns:
#   [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]

if not theory_prior_file.exists():
    available_hdf5 = sorted([p.name for p in Path('.').glob('*.hdf5')])
    print('Available .hdf5 files in current working directory:', available_hdf5)
    raise FileNotFoundError(
        f'Cannot find {theory_prior_file}. '
        'Either move the HDF5 file into the notebook folder, '
        'or set theory_prior_file to the absolute Windows path.'
    )

with h5py.File(theory_prior_file, 'r') as f:
    print('HDF5 top-level keys:', list(f.keys()))
    if 'FDL' in f and isinstance(f['FDL'], h5py.Group) and 'zData' in f['FDL']:
        z_data = np.asarray(f['FDL/zData'])
    elif 'zData' in f:
        z_data = np.asarray(f['zData'])
    else:
        raise KeyError('Could not find z prior. Expected FDL/zData or zData in_
↳the HDF5 file.')

# Keep the same 8D ordering used by x_data:
# [p_e-^mu, p_e+^mu], with p^mu = [px, py, pz, E]
z_data = z_data[:, :8]

assert x_data.ndim == 2 and x_data.shape[1] == 8, f'x_data should have shape_
↳(N, 8), got {x_data.shape}'
assert z_data.ndim == 2 and z_data.shape[1] == 8, f'z_data should have shape_
↳(N, 8), got {z_data.shape}'
assert np.all(np.isfinite(x_data)), 'x_data contains NaN or inf'
assert np.all(np.isfinite(z_data)), 'z_data contains NaN or inf'
```

```

# OTUS training uses unpaired samples from  $p(x)$  and  $p(z)$ . Use the common sample
↪count.
num_samples = min(len(x_data), len(z_data))
x_data = x_data[:num_samples]
z_data = z_data[:num_samples]

x_dim = int(x_data.shape[1])
z_dim = int(z_data.shape[1])

print('Loaded z prior from:', theory_prior_file)
print('z_data shape:', z_data.shape)
print('x_data shape:', x_data.shape)
print('num_samples used:', num_samples)
print('First z row:', z_data[0])

```

```

HDF5 top-level keys: ['FDL']
Loaded z prior from: cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5
z_data shape: (623944, 8)
x_data shape: (623944, 8)
num_samples used: 623944
First z row: [-29.129568 -29.58772  -46.255836  62.1576    34.138885  32.209503
 -22.410378  52.010963]

```

7.1 Pre-training sanity check

Compare the CMS detector-level mass distribution with the MG5 pre-detector prior before training. The MG5 curve should usually be narrower because it does not include detector resolution, electron energy smearing, or reconstruction effects.

```

[40]: from func_utils import Zboson_mass

x_m_before = Zboson_mass(x_data)
z_m_before = Zboson_mass(z_data)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

ax.hist(x_m_before, bins=bins, histtype='step', linewidth=2, density=True,
↪label='CMS x')
ax.hist(z_m_before, bins=bins, histtype='step', linewidth=2, density=True,
↪label='MG5 z prior')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Normalized events')
ax.set_xlim(70, 110)
ax.set_title('Before training: CMS detector-level x vs MG5 pre-detector z')

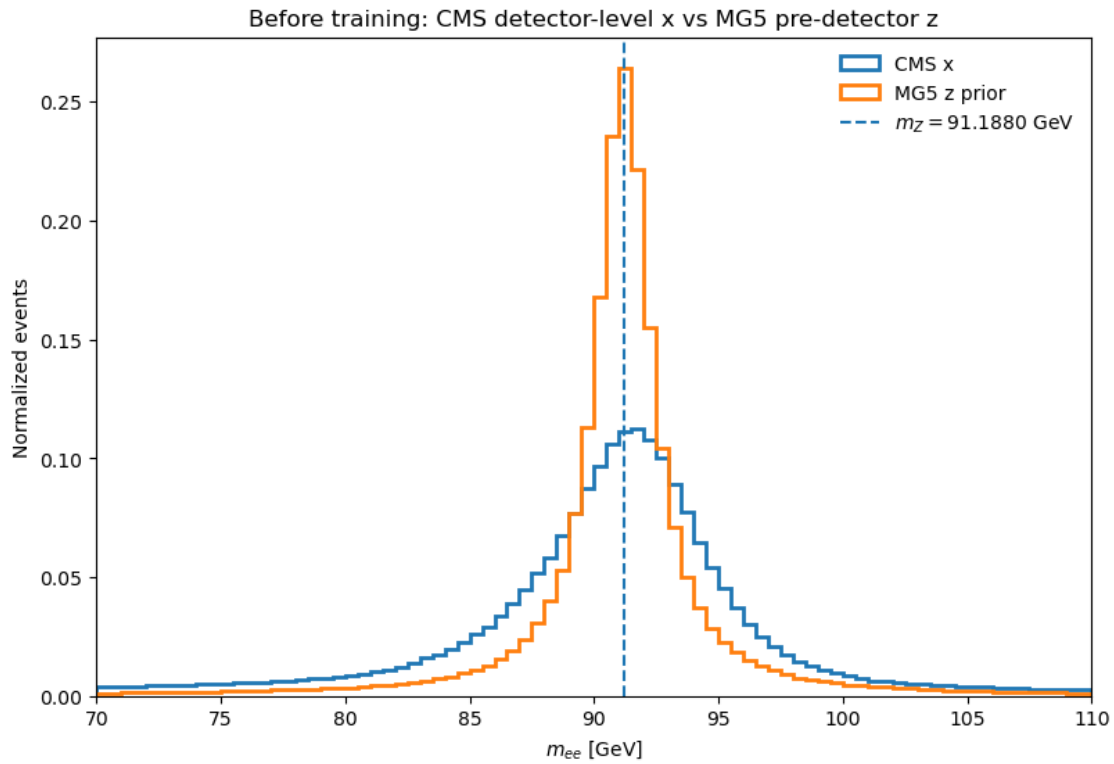
```

```

ax.legend(frameon=False)
plt.show()

print(f'CMS x mass mean: {np.mean(x_m_before):.4f} GeV')
print(f'MG5 z mass mean: {np.mean(z_m_before):.4f} GeV')
print(f'CMS x mass std: {np.std(x_m_before):.4f} GeV')
print(f'MG5 z mass std: {np.std(z_m_before):.4f} GeV')

```



```

CMS x mass mean: 90.6365 GeV
MG5 z mass mean: 91.0889 GeV
CMS x mass std: 5.8531 GeV
MG5 z mass std: 4.0285 GeV

```

7.1.1 Define target invariant masses

```

[41]: # Electrons are treated as massless for consistency with ppzee.ipynb.
x_inv_masses = np.zeros(2)
z_inv_masses = np.zeros(2)

```

8 Train/Test Split

```
[42]: from torch.utils.data import DataLoader

rng = np.random.default_rng(seed)
x_perm = rng.permutation(len(x_data))
z_perm = rng.permutation(len(z_data))
x_data = x_data[x_perm]
z_data = z_data[z_perm]

train_ratio = 0.80
val_ratio = 0.10

train_size = int(num_samples * train_ratio)
val_size = int(num_samples * val_ratio)
test_size = num_samples - train_size - val_size

x_train = x_data[:train_size]
x_val = x_data[train_size:train_size + val_size]
x_test = x_data[train_size + val_size:]

z_train = z_data[:train_size]
z_val = z_data[train_size:train_size + val_size]
z_test = z_data[train_size + val_size:]

x_train, x_val, x_test, z_train, z_val, z_test = list(map(
    lambda arr: arr.astype(float_type),
    [x_train, x_val, x_test, z_train, z_val, z_test],
))

x_train_mean, x_train_std = np.mean(x_train, axis=0), np.std(x_train, axis=0)
z_train_mean, z_train_std = np.mean(z_train, axis=0), np.std(z_train, axis=0)

eval_batch_size = min(20000, max(1, len(x_val)))
eval_loaders = DataLoader(dataset=x_val, batch_size=eval_batch_size,
    ↪shuffle=True), \
    DataLoader(dataset=z_val, batch_size=eval_batch_size,
    ↪shuffle=True)

print('z_train shape, x_train shape:', z_train.shape, x_train.shape)
print('z_val shape, x_val shape:', z_val.shape, x_val.shape)
print('z_test shape, x_test shape:', z_test.shape, x_test.shape)

z_train shape, x_train shape: (499155, 8) (499155, 8)
z_val shape, x_val shape: (62394, 8) (62394, 8)
z_test shape, x_test shape: (62395, 8) (62395, 8)
```

9 Train

9.1 Import Training Specific Libraries and Functions

```
[43]: from torch import optim
      from ppzee_utils import train_and_val
```

9.2 Central hyperparameter block

Edit this one cell when tuning architecture, batch sizes, and staged training.

```
[44]: # =====
# Central Hyperparameter Block
# Edit this cell when tuning the training.
# =====

HP = {
    # -----
    # Model architecture
    # -----
    # Keep the architecture fixed while testing the new loss/stage logic.
    # If the z-cycle version helps but still underfits, then try 3 layers x 512
    ↪ later.
    "cond_noise": True,
    "num_hidden_layers": 2,
    "dim_per_hidden_layer": 256,
    "activation": torch.nn.ReLU,
    "sigma_fun": "softplus",

    "train_batch_size": 4096,
    "eval_batch_size": 20000,
    "preload_data_to_gpu": True,
    "pin_memory_if_cpu_loader": True,

    # -----
    # Training stages
    # -----
    # beta:
    #   x reconstruction weight for x -> z -> x
    #
    # lamb:
    #   z-prior distribution matching weight for E(x_CMS) ~ z_MG5
    #
    # tau:
    #   direct detector-response / generator weight for z_MG5 -> D(z) ~ x_CMS
    #
    # mode = "standard":
```

```

# uses the original OTUS train_and_val loop with the improved
↪z_loss_with_mass.
#
# mode = "z_cycle":
# uses the custom inverse-consistency stage:
# z -> D(z) -> E(D(z)) should recover z.
# This is the stage aimed at fixing the current px/py/pz latent-space
↪problem.
# -----
"stages": [
{
    "name": "stage1_anchor_warmup",
    "mode": "standard",
    "enabled": True,
    "epochs": 50,
    "lr": 1e-3,
    "beta": 1.0,
    "lamb": 1.0,
    "tau": 1.0,
    "rho": 0.0,
    "nu_e": 50.0,
    "nu_d": 50.0,
    "num_slices": 300,
    "log_freq": 10,
    "lr_decay": False,
    "freeze_encoder": False,
    "freeze_decoder": False,
},
{
    "name": "stage2_joint_transport",
    "mode": "standard",
    "enabled": True,
    "epochs": 50,
    "lr": 5e-4,
    "beta": 1.0,
    "lamb": 1.0,
    "tau": 1.0,
    "rho": 0.0,
    "nu_e": 0.0,
    "nu_d": 0.0,
    "num_slices": 300,
    "log_freq": 20,
    "lr_decay": True,
    "freeze_encoder": False,
    "freeze_decoder": False,
},
{

```

```

    "name": "stage3_decoder_response",
    "mode": "standard",
    "enabled": True,
    "epochs": 50,
    "lr": 1e-4,
    "beta": 0.5,
    "lamb": 0.8,
    "tau": 3.0,
    "rho": 0.0,
    "nu_e": 0.0,
    "nu_d": 0.0,
    "num_slices": 500,
    "log_freq": 20,
    "lr_decay": True,
    "freeze_encoder": False,
    "freeze_decoder": False,
},
{
    "name": "stage4_encoder_distribution_polish",
    "mode": "standard",
    "enabled": True,
    "epochs": 50,
    "lr": 3e-5,
    "beta": 1.0,
    "lamb": 4.0,
    "tau": 0.0,
    "rho": 0.0,
    "nu_e": 0.0,
    "nu_d": 0.0,
    "num_slices": 500,
    "log_freq": 10,
    "lr_decay": True,
    "freeze_encoder": False,
    "freeze_decoder": True,
},
{
    "name": "stage5_z_cycle_inverse_polish",
    "mode": "z_cycle",
    "enabled": True,
    "epochs": 50,
    "lr": 1e-5,

    # In z-cycle mode:
    # beta = x -> z -> x reconstruction weight
    # lamb = E(x_CMS) ~ z_MG5 prior-matching weight
    "beta": 1.0,
    "lamb": 2.0,

```

```

        "tau": 0.0,

        # New inverse-consistency terms used only in mode="z_cycle"
        "z_cycle_weight": 8.0,
        "transverse_weight": 4.0,

        "rho": 0.0,
        "nu_e": 0.0,
        "nu_d": 0.0,
        "num_slices": 500,
        "log_freq": 10,
        "lr_decay": False,

        # Freeze decoder because z -> x already looks promising.
        # Train the encoder to become a better inverse of the learned
        ↪decoder.
        "freeze_encoder": False,
        "freeze_decoder": True,
    },
    {
        "name": "stage6_optional_balanced_finetune",
        "mode": "standard",
        "enabled": False,
        "epochs": 40,
        "lr": 5e-6,
        "beta": 1.0,
        "lamb": 1.0,
        "tau": 0.5,
        "rho": 0.0,
        "nu_e": 0.0,
        "nu_d": 0.0,
        "num_slices": 1200,
        "log_freq": 10,
        "lr_decay": False,
        "freeze_encoder": False,
        "freeze_decoder": False,
    },
],
}

if HP["cond_noise"]:
    from models import CondNoiseAutoencoder
    Autoencoder = CondNoiseAutoencoder
else:
    from models import Autoencoder

```

```

print('Using model class:', Autoencoder.__name__)
print('Enabled stages:', [s['name'] for s in HP['stages'] if s.get('enabled',
↪True)])
print('train_batch_size:', HP['train_batch_size'])
print('preload_data_to_gpu:', HP['preload_data_to_gpu'])

```

```

Using model class: CondNoiseAutoencoder
Enabled stages: ['stage1_anchor_warmup', 'stage2_joint_transport',
'stage3_decoder_response', 'stage4_encoder_distribution_polish',
'stage5_z_cycle_inverse_polish']
train_batch_size: 4096
preload_data_to_gpu: True

```

9.3 Build model from hyperparameter block

```

[45]: # =====
# Build Model from HP
# =====

num_hidden_layers = HP["num_hidden_layers"]
dim_per_hidden_layer = HP["dim_per_hidden_layer"]
hidden_layer_dims = num_hidden_layers * [dim_per_hidden_layer]

activation = HP["activation"]
sigma_fun = HP["sigma_fun"]

model = Autoencoder(
    x_dim=x_dim,
    z_dim=z_dim,
    hidden_layer_dims=hidden_layer_dims,
    raw_io=True,
    x_stats=np.stack([x_train_mean, x_train_std]),
    z_stats=np.stack([z_train_mean, z_train_std]),
    x_inv_masses=x_inv_masses,
    z_inv_masses=z_inv_masses,
    stoch_enc=True,
    stoch_dec=True,
    activation=activation,
    sigma_fun=sigma_fun,
)

model

```

```

[45]: CondNoiseAutoencoder(
  (encoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(

```

```

        (0): Linear(in_features=14, out_features=256, bias=True)
        (1): ReLU()
        (2): Linear(in_features=256, out_features=256, bias=True)
        (3): ReLU()
        (4): Linear(in_features=256, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
      (0): Linear(in_features=8, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=12, bias=True)
    )
  )
  (decoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(
      (0): Linear(in_features=14, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
      (0): Linear(in_features=8, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=12, bias=True)
    )
  )
)
)

```

```

[46]: from func_utils import sliced_wd
import torch

P = 2
EPS = 1e-6

# -----
# Internal z-loss component weights
# -----
lambda_z8 = 0.40
lambda_marginal = 1.50
lambda_mass = 0.70
lambda_phys = 0.50
lambda_transverse = 2.50

```

```

lambda_longitudinal = 1.20
lambda_tail = 1.00

def to_like(value, ref):
    if isinstance(value, torch.Tensor):
        return value.detach().to(dtype=ref.dtype, device=ref.device)
    return torch.as_tensor(value, dtype=ref.dtype, device=ref.device)

def first_tensor(y):
    """Support model methods that return either a tensor or a tuple/list."""
    if isinstance(y, (tuple, list)):
        return y[0]
    return y

def wasserstein_1d_sorted(a, b, p=2):
    """
    1D unpaired Wasserstein-style loss.
    Sort both distributions and compare quantiles.
    """
    a = torch.sort(a.reshape(-1))[0]
    b = torch.sort(b.reshape(-1))[0]

    n = min(a.numel(), b.numel())
    a = a[:n]
    b = b[:n]

    return torch.mean(torch.abs(a - b) ** p)

def weighted_marginal_wd(a, b, weights, p=2):
    """
    Apply 1D sorted Wasserstein to each feature separately.

    This is important because a random sliced-Wasserstein loss can miss a
    badly matched single coordinate, especially  $p_x/p_y/p_z$ .
    """
    weights = to_like(weights, a)

    loss = a.new_tensor(0.0)
    for j in range(a.shape[1]):
        loss = loss + weights[j] * wasserstein_1d_sorted(a[:, j], b[:, j], p=p)

    return loss / torch.sum(weights)

```

```

def tail_wasserstein_abs(a, b, tail_frac=0.20, p=2):
    """
    Compare the high-/variable/ tails.

    This fights the failure mode where the encoded distribution becomes too
    narrow or shifts too many events toward the most-probable value.
    """
    a = torch.sort(torch.abs(a.reshape(-1)))[0]
    b = torch.sort(torch.abs(b.reshape(-1)))[0]

    n = min(a.numel(), b.numel())
    a = a[:n]
    b = b[:n]

    start = int((1.0 - tail_frac) * n)
    start = max(0, min(start, n - 1))

    return torch.mean(torch.abs(a[start:] - b[start:]) ** p)

def standardize_x_raw(x):
    mean = to_like(x_train_mean, x)
    std = to_like(x_train_std, x)
    return (x - mean) / (std + EPS)

def standardize_z_raw(z):
    mean = to_like(z_train_mean, z)
    std = to_like(z_train_std, z)
    return (z - mean) / (std + EPS)

def paired_mse_standardized(a, b, standardize_fun):
    """Paired event-by-event MSE after standardization."""
    a_s = standardize_fun(a)
    b_s = standardize_fun(b)
    return torch.mean((a_s - b_s) ** 2)

def dilepton_mass_torch(z, eps=EPS):
    """
    z columns:
    [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]
    """
    px = z[:, 0] + z[:, 4]
    py = z[:, 1] + z[:, 5]

```

```

pz = z[:, 2] + z[:, 6]
E = z[:, 3] + z[:, 7]

m2 = E**2 - px**2 - py**2 - pz**2
return torch.sqrt(torch.clamp(m2, min=eps))

def safe_eta_from_pt_pz(pt, pz, eps=EPS):
    """
    eta = asinh(pz / pt), written without torch.asinh for compatibility.
    """
    p = torch.sqrt(pt**2 + pz**2 + eps)
    return torch.log(torch.clamp(p + pz, min=eps) / (pt + eps))

def z_physics_features(z, eps=EPS):
    """
    General physics features:
    [pt1, eta1, pt2, eta2, m11, pt11, y11, cos_dphi, sin_dphi]
    """
    px1, py1, pz1, E1 = z[:, 0], z[:, 1], z[:, 2], z[:, 3]
    px2, py2, pz2, E2 = z[:, 4], z[:, 5], z[:, 6], z[:, 7]

    pt1 = torch.sqrt(px1**2 + py1**2 + eps)
    pt2 = torch.sqrt(px2**2 + py2**2 + eps)

    eta1 = safe_eta_from_pt_pz(pt1, pz1, eps=eps)
    eta2 = safe_eta_from_pt_pz(pt2, pz2, eps=eps)

    px = px1 + px2
    py = py1 + py2
    pz = pz1 + pz2
    E = E1 + E2

    m2 = E**2 - px**2 - py**2 - pz**2
    m11 = torch.sqrt(torch.clamp(m2, min=eps))

    pt11 = torch.sqrt(px**2 + py**2 + eps)

    y_num = torch.clamp(E + pz, min=eps)
    y_den = torch.clamp(E - pz, min=eps)
    y11 = 0.5 * torch.log(y_num / y_den)

    dot = px1 * px2 + py1 * py2
    cross = px1 * py2 - py1 * px2

    cos_dphi = torch.clamp(dot / (pt1 * pt2 + eps), min=-1.0, max=1.0)

```

```

sin_dphi = torch.clamp(cross / (pt1 * pt2 + eps), min=-1.0, max=1.0)

return torch.stack(
    [pt1, eta1, pt2, eta2, m11, pt11, y11, cos_dphi, sin_dphi],
    dim=1
)

def z_transverse_features(z, eps=EPS):
    """
    px/py-focused features.

    Features:
    [px1, py1, px2, py2, pt1, pt2, px_sum, py_sum, pt11,
     px_diff, py_diff, cos_dphi, sin_dphi]
    """
    px1, py1 = z[:, 0], z[:, 1]
    px2, py2 = z[:, 4], z[:, 5]

    pt1 = torch.sqrt(px1**2 + py1**2 + eps)
    pt2 = torch.sqrt(px2**2 + py2**2 + eps)

    px_sum = px1 + px2
    py_sum = py1 + py2
    pt11 = torch.sqrt(px_sum**2 + py_sum**2 + eps)

    px_diff = px1 - px2
    py_diff = py1 - py2

    dot = px1 * px2 + py1 * py2
    cross = px1 * py2 - py1 * px2

    cos_dphi = torch.clamp(dot / (pt1 * pt2 + eps), min=-1.0, max=1.0)
    sin_dphi = torch.clamp(cross / (pt1 * pt2 + eps), min=-1.0, max=1.0)

    return torch.stack(
        [px1, py1, px2, py2, pt1, pt2, px_sum, py_sum, pt11,
         px_diff, py_diff, cos_dphi, sin_dphi],
        dim=1
    )

def z_longitudinal_features(z, eps=EPS):
    """
    pz-focused features.

    Features:

```

```

    [pz1, pz2, pz_sum, pz_diff, eta1, eta2, yll]
    """
    px1, py1, pz1, E1 = z[:, 0], z[:, 1], z[:, 2], z[:, 3]
    px2, py2, pz2, E2 = z[:, 4], z[:, 5], z[:, 6], z[:, 7]

    pt1 = torch.sqrt(px1**2 + py1**2 + eps)
    pt2 = torch.sqrt(px2**2 + py2**2 + eps)

    eta1 = safe_eta_from_pt_pz(pt1, pz1, eps=eps)
    eta2 = safe_eta_from_pt_pz(pt2, pz2, eps=eps)

    pz_sum = pz1 + pz2
    pz_diff = pz1 - pz2

    E = E1 + E2
    y_num = torch.clamp(E + pz_sum, min=eps)
    y_den = torch.clamp(E - pz_sum, min=eps)
    yll = 0.5 * torch.log(y_num / y_den)

    return torch.stack(
        [pz1, pz2, pz_sum, pz_diff, eta1, eta2, yll],
        dim=1
    )

# =====
# Precompute normalization constants from MG5 z_train prior
# =====

with torch.no_grad():
    z_train_torch = torch.as_tensor(z_train, dtype=torch.float32)

    m_z_train = dilepton_mass_torch(z_train_torch)
    m_z_mean = m_z_train.mean().detach()
    m_z_std = m_z_train.std().detach()

    fz_train = z_physics_features(z_train_torch)
    fz_mean = fz_train.mean(dim=0).detach()
    fz_std = fz_train.std(dim=0).detach()

    trans_train = z_transverse_features(z_train_torch)
    trans_mean = trans_train.mean(dim=0).detach()
    trans_std = trans_train.std(dim=0).detach()

    long_train = z_longitudinal_features(z_train_torch)
    long_mean = long_train.mean(dim=0).detach()
    long_std = long_train.std(dim=0).detach()

```

```

print("MG5 z mass mean/std:", m_z_mean, m_z_std)
print("MG5 transverse feature mean/std shapes:", trans_mean.shape, trans_std.
      ↪shape)
print("MG5 longitudinal feature mean/std shapes:", long_mean.shape, long_std.
      ↪shape)

def standardize_mass(m, ref=None):
    # ref is kept for backward compatibility with existing calls,
    # but m is the correct dtype/device reference.
    mean = to_like(m_z_mean, m)
    std = to_like(m_z_std, m)
    return (m - mean) / (std + EPS)

def standardize_features(f, mean, std):
    mean = to_like(mean, f)
    std = to_like(std, f)
    return (f - mean) / (std + EPS)

def z_channel_marginal_loss(z, z_tilde):
    """
    Direct per-channel marginal loss on standardized raw z variables.

    Channel order:
    [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]

    px/py are currently the largest failure, pz is secondary,
    and E is already relatively reasonable.
    """
    z_s = standardize_z_raw(z)
    zt_s = standardize_z_raw(z_tilde)

    channel_weights = [
        6.0, 6.0, 3.0, 0.5,
        6.0, 6.0, 3.0, 0.5,
    ]

    return weighted_marginal_wd(z_s, zt_s, channel_weights, p=P)

def z_transverse_loss(z, z_tilde):
    """
    Explicit px/py correction loss.
    Used both inside z_loss_with_mass and again in z-cycle polish.

```

```

"""
f_z = standardize_features(z_transverse_features(z), trans_mean, trans_std)
f_zt = standardize_features(z_transverse_features(z_tilde), trans_mean,
↳trans_std)

# Feature order:
# [px1, py1, px2, py2, pt1, pt2, px_sum, py_sum, ptll,
#  px_diff, py_diff, cos_dphi, sin_dphi]
weights = [
    5.0, 5.0, 5.0, 5.0,
    2.0, 2.0,
    3.0, 3.0, 2.0,
    3.0, 3.0,
    1.0, 1.0,
]

loss_marginal = weighted_marginal_wd(f_z, f_zt, weights, p=P)
loss_joint = sliced_wd(f_z, f_zt, config.get("num_slices", 1000), P)

z_s = standardize_z_raw(z)
zt_s = standardize_z_raw(z_tilde)

loss_tail = (
    tail_wasserstein_abs(z_s[:, 0], zt_s[:, 0], tail_frac=0.20, p=P)
    + tail_wasserstein_abs(z_s[:, 1], zt_s[:, 1], tail_frac=0.20, p=P)
    + tail_wasserstein_abs(z_s[:, 4], zt_s[:, 4], tail_frac=0.20, p=P)
    + tail_wasserstein_abs(z_s[:, 5], zt_s[:, 5], tail_frac=0.20, p=P)
) / 4.0

return loss_marginal + 0.5 * loss_joint + 0.5 * loss_tail

def z_longitudinal_loss(z, z_tilde):
    """
    Explicit pz/eta/rapidity correction loss.
    """
    f_z = standardize_features(z_longitudinal_features(z), long_mean, long_std)
    f_zt = standardize_features(z_longitudinal_features(z_tilde), long_mean,
↳long_std)

    # Feature order:
    # [pz1, pz2, pz_sum, pz_diff, eta1, eta2, yll]
    weights = [
        4.0, 4.0, 3.0, 2.0, 2.0, 2.0, 2.0
    ]

    loss_marginal = weighted_marginal_wd(f_z, f_zt, weights, p=P)

```

```

loss_joint = sliced_wd(f_z, f_zt, config.get("num_slices", 1000), P)

z_s = standardize_z_raw(z)
zt_s = standardize_z_raw(z_tilde)

loss_tail = (
    tail_wasserstein_abs(z_s[:, 2], zt_s[:, 2], tail_frac=0.20, p=P)
    + tail_wasserstein_abs(z_s[:, 6], zt_s[:, 6], tail_frac=0.20, p=P)
) / 2.0

return loss_marginal + 0.5 * loss_joint + 0.5 * loss_tail

def z_loss_with_mass(z, z_tilde):
    """
    Improved z-space loss for the standard OTUS stages.

    z:
        MG5 truth-level prior batch

    z_tilde:
        encoded CMS batch, z_tilde = E(x)

    Important:
        This is still distribution-level matching.
        It is not event-by-event truth matching.
    """
    num_slices = config.get("num_slices", 1000)

    z_std = standardize_z_raw(z)
    zt_std = standardize_z_raw(z_tilde)

    # 1. Global standardized 8D sliced-Wasserstein
    loss_z_8d = sliced_wd(z_std, zt_std, num_slices, P)

    # 2. Direct raw channel marginal loss
    loss_marginal = z_channel_marginal_loss(z, z_tilde)

    # 3. Mass loss
    m_z = standardize_mass(dilepton_mass_torch(z), z)
    m_zt = standardize_mass(dilepton_mass_torch(z_tilde), z_tilde)
    loss_mass = wasserstein_1d_sorted(m_z, m_zt, p=P)

    # 4. General physics-feature loss
    f_z = standardize_features(z_physics_features(z), fz_mean, fz_std)
    f_zt = standardize_features(z_physics_features(z_tilde), fz_mean, fz_std)
    loss_phys = sliced_wd(f_z, f_zt, num_slices, P)

```

```

# 5. px/py-focused loss
loss_transverse = z_transverse_loss(z, z_tilde)

# 6. pz-focused loss
loss_longitudinal = z_longitudinal_loss(z, z_tilde)

# 7. General high-|z| tail loss over all 8 standardized channels
loss_tail = standardize_z_raw(z).new_tensor(0.0)
for j in range(8):
    loss_tail = loss_tail + tail_wasserstein_abs(z_std[:, j], zt_std[:, j],
tail_frac=0.20, p=P)
loss_tail = loss_tail / 8.0

return (
    lambda_z8 * loss_z_8d
    + lambda_marginal * loss_marginal
    + lambda_mass * loss_mass
    + lambda_phys * loss_phys
    + lambda_transverse * loss_transverse
    + lambda_longitudinal * loss_longitudinal
    + lambda_tail * loss_tail
)

```

MG5 z mass mean/std: tensor(91.0878) tensor(4.0349)

MG5 transverse feature mean/std shapes: torch.Size([13]) torch.Size([13])

MG5 longitudinal feature mean/std shapes: torch.Size([7]) torch.Size([7])

9.4 Train OTUS

```

[47]: # =====
# Fast DataLoaders from HP
# =====
# The old version used PyTorch DataLoader directly on NumPy arrays.
# That works, but it creates/collates CPU tensors batch-by-batch.
# For this tabular 8D dataset, it is faster to make torch tensors once.
# If CUDA is available, we keep train/val tensors on the GPU once.
# Precision is unchanged because torch.as_tensor keeps the array dtype.

class FastTensorLoader:
    """Minimal batch loader for one tensor.

    It is compatible with the OTUS train_and_val code as long as the code only
    needs iteration and len(loader). It returns one tensor batch at a time.
    """
    def __init__(self, tensor, batch_size, shuffle=True, drop_last=False):
        self.tensor = tensor

```

```

        self.batch_size = int(batch_size)
        self.shuffle = bool(shuffle)
        self.drop_last = bool(drop_last)
        self.n = int(tensor.shape[0])
        self.dataset = tensor # for compatibility with code that checks loader.
↪dataset

    def __len__(self):
        if self.drop_last:
            return self.n // self.batch_size
        return (self.n + self.batch_size - 1) // self.batch_size

    def __iter__(self):
        if self.shuffle:
            idx = torch.randperm(self.n, device=self.tensor.device)
        else:
            idx = torch.arange(self.n, device=self.tensor.device)

        stop = self.n if not self.drop_last else (self.n // self.batch_size) * ↵
↪self.batch_size
        for start in range(0, stop, self.batch_size):
            batch_idx = idx[start:start + self.batch_size]
            yield self.tensor.index_select(0, batch_idx)

def make_training_tensor(arr, name, loader_device, pin_cpu_memory=False):
    t = torch.as_tensor(arr) # preserve dtype from float_type; no precision ↵
↪change
    if loader_device.type == 'cuda':
        t = t.to(loader_device)
    elif pin_cpu_memory and torch.cuda.is_available():
        t = t.pin_memory()
    print(f'{name}: shape={tuple(t.shape)}, dtype={t.dtype}, device={t.device}')
    return t

train_batch_size = min(HP["train_batch_size"], max(1, len(x_train)))
eval_batch_size = min(HP["eval_batch_size"], max(1, len(x_val)))

device_obj = torch.device(device)
preload_to_gpu = bool(HP.get("preload_data_to_gpu", True)) and (device_obj.type ↵
↪== 'cuda')
loader_device = device_obj if preload_to_gpu else torch.device('cpu')
pin_cpu_memory = bool(HP.get("pin_memory_if_cpu_loader", True)) and ↵
↪(loader_device.type == 'cpu') and torch.cuda.is_available()

print('loader_device:', loader_device)
print('pin_cpu_memory:', pin_cpu_memory)

```

```

x_train_loader = make_training_tensor(x_train, 'x_train_loader', loader_device,
    ↪pin_cpu_memory)
z_train_loader = make_training_tensor(z_train, 'z_train_loader', loader_device,
    ↪pin_cpu_memory)
x_val_loader = make_training_tensor(x_val, 'x_val_loader', loader_device,
    ↪pin_cpu_memory)
z_val_loader = make_training_tensor(z_val, 'z_val_loader', loader_device,
    ↪pin_cpu_memory)

train_loaders = (
    FastTensorLoader(x_train_loader, batch_size=train_batch_size, shuffle=True),
    FastTensorLoader(z_train_loader, batch_size=train_batch_size, shuffle=True),
)

eval_loaders = (
    FastTensorLoader(x_val_loader, batch_size=eval_batch_size, shuffle=False),
    FastTensorLoader(z_val_loader, batch_size=eval_batch_size, shuffle=False),
)

print('train_batch_size:', train_batch_size)
print('eval_batch_size:', eval_batch_size)
print('training batches per epoch:', min(len(train_loaders[0]),
    ↪len(train_loaders[1])))
print('eval batches:', min(len(eval_loaders[0]), len(eval_loaders[1])))

```

```

loader_device: cuda
pin_cpu_memory: False
x_train_loader: shape=(499155, 8), dtype=torch.float32, device=cuda:0
z_train_loader: shape=(499155, 8), dtype=torch.float32, device=cuda:0
x_val_loader: shape=(62394, 8), dtype=torch.float32, device=cuda:0
z_val_loader: shape=(62394, 8), dtype=torch.float32, device=cuda:0
train_batch_size: 4096
eval_batch_size: 20000
training batches per epoch: 122
eval batches: 4

```

```

[48]: %%time

# =====
# Train OTUS using stages from HP
# =====
#
# Standard stages:
#   call the original OTUS train_and_val with z_loss_with_mass.
#
# z_cycle stage:

```

```

#   custom inverse-consistency polish:
#       real CMS:       $x \rightarrow E(x) \rightarrow D(E(x))$  should reconstruct  $x$ 
#       distribution:  $E(x_{CMS})$  should match  $z_{MG5}$ 
#       synthetic:      $z \rightarrow D(z) \rightarrow E(D(z))$  should reconstruct  $z$ 
#       targeted:       $p_x/p_y$ -focused transverse  $z$  loss
# =====

history = None
eval_losses = None

device_obj = torch.device(device)
model.to(device_obj)

def set_trainable(model, train_encoder=True, train_decoder=True):
    for p in model.encoder.parameters():
        p.requires_grad_(train_encoder)
    for p in model.decoder.parameters():
        p.requires_grad_(train_decoder)

def make_stage_config(stage):
    return {
        "num_hidden_layers": HP["num_hidden_layers"],
        "dim_per_hidden_layer": HP["dim_per_hidden_layer"],
        "epochs": stage["epochs"],
        "lr": stage["lr"],
        "beta": stage["beta"],
        "lamb": stage["lamb"],
        "tau": stage["tau"],
        "rho": stage["rho"],
        "nu_e": stage["nu_e"],
        "nu_d": stage["nu_d"],
        "num_slices": stage["num_slices"],
    }

def ensure_history(history):
    if history is None:
        history = {}

    for key in ["epoch", "train_loss", "train_x_loss", "train_z_loss", "
    ↪eval_loss"]:
        history.setdefault(key, [])
        if not isinstance(history[key], list):
            history[key] = list(history[key])

```

```

    return history

def as_float(x):
    if isinstance(x, torch.Tensor):
        return float(x.detach().cpu())
    return float(x)

def last_epoch_from_history(history):
    if history is None or "epoch" not in history or len(history["epoch"]) == 0:
        return 0
    vals = []
    for v in history["epoch"]:
        if isinstance(v, torch.Tensor):
            vals.append(float(v.detach().cpu()))
        else:
            vals.append(float(v))
    return int(max(vals))

def append_history(history, epoch, train_loss, train_x_loss, train_z_loss,
    eval_loss):
    history = ensure_history(history)
    history["epoch"].append(epoch)
    history["train_loss"].append(as_float(train_loss))
    history["train_x_loss"].append(as_float(train_x_loss))
    history["train_z_loss"].append(as_float(train_z_loss))
    history["eval_loss"].append(as_float(eval_loss))
    return history

def z_cycle_losses_for_batch(model, x_batch, z_batch, stage):
    """
    Compute losses for the custom inverse-consistency stage.

    Decoder can be frozen, but gradients still flow through decoder operations
    into the encoder for  $x \rightarrow z \rightarrow x$  reconstruction.
    """
    beta = stage.get("beta", 1.0)
    lamb = stage.get("lamb", 1.0)
    z_cycle_weight = stage.get("z_cycle_weight", 0.0)
    transverse_weight = stage.get("transverse_weight", 0.0)

    # -----
    # 1. Real CMS reconstruction:
    #  $x \rightarrow E(x) \rightarrow D(E(x))$  should recover  $x$ 

```

```

# -----
z_from_x = first_tensor(model.encode(x_batch))
x_reco = first_tensor(model.decode(z_from_x))

loss_x_reco = paired_mse_standardized(
    x_reco,
    x_batch,
    standardize_x_raw,
)

# -----
# 2. Prior distribution matching:
#    $E(x_{CMS})$  should look like MG5  $z$ 
# -----
loss_z_prior = z_loss_with_mass(z_batch, z_from_x)

# -----
# 3. Synthetic paired z-cycle:
#    $z \rightarrow D(z) \rightarrow E(D(z))$  should recover the same  $z$ 
# -----
x_from_z = first_tensor(model.decode(z_batch))
z_cycle = first_tensor(model.encode(x_from_z))

loss_z_cycle = paired_mse_standardized(
    z_cycle,
    z_batch,
    standardize_z_raw,
)

# -----
# 4. Targeted transverse correction:
#   Directly pushes  $p_x/p_y$  features of  $E(x_{CMS})$  toward MG5  $z$ .
# -----
loss_transverse = z_transverse_loss(z_batch, z_from_x)

# -----
# Total
# -----
loss_total = (
    beta * loss_x_reco
    + lamb * loss_z_prior
    + z_cycle_weight * loss_z_cycle
    + transverse_weight * loss_transverse
)

return {
    "total": loss_total,

```

```

        "x_reco": loss_x_reco,
        "z_prior": loss_z_prior,
        "z_cycle": loss_z_cycle,
        "transverse": loss_transverse,
    }

def run_z_cycle_stage(model, train_loaders, eval_loaders, stage, history):
    """
    Custom stage for inverse consistency.

    This is intentionally outside ppzee_utils.train_and_val because the
    original OTUS loop does not include  $z \rightarrow x \rightarrow z$ .
    """
    global config

    config = make_stage_config(stage)

    freeze_encoder = stage.get("freeze_encoder", False)
    freeze_decoder = stage.get("freeze_decoder", False)

    set_trainable(
        model,
        train_encoder=not freeze_encoder,
        train_decoder=not freeze_decoder,
    )

    trainable_params = [p for p in model.parameters() if p.requires_grad]
    optimizer = optim.Adam(trainable_params, lr=stage["lr"])

    history = ensure_history(history)
    start_epoch = last_epoch_from_history(history)

    print("Custom z-cycle stage")
    print("Train encoder:", not freeze_encoder)
    print("Train decoder:", not freeze_decoder)
    print("z_cycle_weight:", stage.get("z_cycle_weight", 0.0))
    print("transverse_weight:", stage.get("transverse_weight", 0.0))

    final_eval = None

    for local_epoch in range(1, stage["epochs"] + 1):
        model.train()

        sums = {
            "total": 0.0,
            "x_reco": 0.0,

```

```

        "z_prior": 0.0,
        "z_cycle": 0.0,
        "transverse": 0.0,
    }
    nbatches = 0

    for x_batch, z_batch in zip(train_loaders[0], train_loaders[1]):
        x_batch = x_batch.to(device_obj)
        z_batch = z_batch.to(device_obj)

        optimizer.zero_grad()

        losses = z_cycle_losses_for_batch(model, x_batch, z_batch, stage)
        losses["total"].backward()

        torch.nn.utils.clip_grad_norm_(trainable_params, max_norm=10.0)
        optimizer.step()

        for key in sums:
            sums[key] += as_float(losses[key])
        nbatches += 1

    for key in sums:
        sums[key] /= max(1, nbatches)

    should_log = (
        local_epoch == 1
        or local_epoch == stage["epochs"]
        or local_epoch % stage["log_freq"] == 0
    )

    if should_log:
        model.eval()

        eval_sums = {
            "total": 0.0,
            "x_reco": 0.0,
            "z_prior": 0.0,
            "z_cycle": 0.0,
            "transverse": 0.0,
        }
        eval_batches = 0

        with torch.no_grad():
            for x_eval, z_eval in zip(eval_loaders[0], eval_loaders[1]):
                x_eval = x_eval.to(device_obj)
                z_eval = z_eval.to(device_obj)

```

```

        eval_losses_batch = z_cycle_losses_for_batch(model, x_eval,
↪z_eval, stage)

        for key in eval_sums:
            eval_sums[key] += as_float(eval_losses_batch[key])
            eval_batches += 1

    for key in eval_sums:
        eval_sums[key] /= max(1, eval_batches)

    absolute_epoch = start_epoch + local_epoch

    # For the standard history plot:
    #   train_x_loss = x reconstruction
    #   train_z_loss = z-prior + z-cycle + transverse terms
    history = append_history(
        history,
        epoch=absolute_epoch,
        train_loss=sums["total"],
        train_x_loss=sums["x_reco"],
        train_z_loss=(
            sums["z_prior"]
            + sums["z_cycle"]
            + sums["transverse"]
        ),
        eval_loss=eval_sums["total"],
    )

    final_eval = {
        "loss": eval_sums["total"],
        "x_reco": eval_sums["x_reco"],
        "z_prior": eval_sums["z_prior"],
        "z_cycle": eval_sums["z_cycle"],
        "transverse": eval_sums["transverse"],
    }

    print(
        f"epoch {absolute_epoch:04d} "
        f"(local {local_epoch:04d}) | "
        f"train_total={sums['total']:.4e} | "
        f"train_x={sums['x_reco']:.4e} | "
        f"train_zprior={sums['z_prior']:.4e} | "
        f"train_zcycle={sums['z_cycle']:.4e} | "
        f"train_trans={sums['transverse']:.4e} | "
        f"eval_total={eval_sums['total']:.4e}"
    )

```

```

    return final_eval, history

for stage in HP["stages"]:
    if not stage.get("enabled", True):
        print(f"Skipping disabled stage: {stage['name']}")
        continue

    print("\n" + "=" * 80)
    print("Starting training stage:", stage["name"])
    print("Mode:", stage.get("mode", "standard"))
    print("=" * 80)

    mode = stage.get("mode", "standard")

    if mode == "standard":
        freeze_encoder = stage.get("freeze_encoder", False)
        freeze_decoder = stage.get("freeze_decoder", False)

        set_trainable(
            model,
            train_encoder=not freeze_encoder,
            train_decoder=not freeze_decoder,
        )

        print("Train encoder:", not freeze_encoder)
        print("Train decoder:", not freeze_decoder)

        config = make_stage_config(stage)

        trainable_params = [p for p in model.parameters() if p.requires_grad]
        optimizer = optim.Adam(trainable_params, lr=config["lr"])

        eval_losses, history = train_and_val(
            model,
            train_loaders,
            eval_loaders,
            config,
            optimizer,
            verbose=True,
            prev_hist=history,
            log_freq=stage["log_freq"],
            lr_decay=stage["lr_decay"],
            z_loss_fun=z_loss_with_mass,
        )

```

```

elif mode == "z_cycle":
    eval_losses, history = run_z_cycle_stage(
        model,
        train_loaders,
        eval_loaders,
        stage,
        history,
    )

else:
    raise ValueError(f"Unknown training stage mode: {mode}")

# Unfreeze everything after training, useful before saving / future fine-tuning
set_trainable(model, train_encoder=True, train_decoder=True)

print("\nTraining complete.")
print("Final eval losses:", eval_losses)

```

```

=====
Starting training stage: stage1_anchor_warmup
Mode: standard
=====
Train encoder: True
Train decoder: True
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 50, 'lr': 0.001,
'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 50.0, 'nu_d': 50.0,
'num_slices': 300}
epoch: 0
train -- loss:200.307, x_loss:103.31, z_loss:15.22, alt_x_loss:15.7759,
x_constraint_loss:0, anchor_loss:1.32003
eval -- loss:2225.61, x_loss:98.4979, z_loss:15.1859, alt_x_loss:2210.42,
anchor_loss:1.13075
epoch: 10
train -- loss:17.8015, x_loss:4.80031, z_loss:6.37525, alt_x_loss:0.865372,
x_constraint_loss:0, anchor_loss:0.115212
eval -- loss:61.6458, x_loss:4.14419, z_loss:5.93714, alt_x_loss:55.7087,
anchor_loss:0.201007
epoch: 20
train -- loss:9.25797, x_loss:2.3401, z_loss:3.52434, alt_x_loss:0.892521,
x_constraint_loss:0, anchor_loss:0.0500201
eval -- loss:43.5765, x_loss:2.19868, z_loss:3.57652, alt_x_loss:40,
anchor_loss:0.0773718
epoch: 30
train -- loss:6.43819, x_loss:2.01206, z_loss:2.3256, alt_x_loss:0.765227,
x_constraint_loss:0, anchor_loss:0.0267062
eval -- loss:42.6941, x_loss:2.19736, z_loss:2.31097, alt_x_loss:40.3831,
anchor_loss:0.0381341

```

```

epoch: 40
train -- loss:4.71064, x_loss:1.6974, z_loss:1.69988, alt_x_loss:0.391402,
x_constraint_loss:0, anchor_loss:0.0184392
eval -- loss:35.3554, x_loss:1.6673, z_loss:1.7734, alt_x_loss:33.582,
anchor_loss:0.025473
epoch: 49
train -- loss:4.16867, x_loss:1.7068, z_loss:1.4278, alt_x_loss:0.363756,
x_constraint_loss:0, anchor_loss:0.0134063
eval -- loss:32.0775, x_loss:1.87007, z_loss:1.46302, alt_x_loss:30.6145,
anchor_loss:0.0201249

```

```

=====
Starting training stage: stage2_joint_transport

```

```

Mode: standard
=====

```

```

Train encoder: True
Train decoder: True
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 50, 'lr':
0.0005, 'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
0.0, 'num_slices': 300}
epoch: 0
train -- loss:3.65413, x_loss:1.15316, z_loss:1.84662, alt_x_loss:0.654349,
x_constraint_loss:0, anchor_loss:0
eval -- loss:32.9062, x_loss:1.20983, z_loss:1.55107, alt_x_loss:31.3552,
anchor_loss:0.0230716
epoch: 20
train -- loss:2.45763, x_loss:0.880846, z_loss:1.09082, alt_x_loss:0.485967,
x_constraint_loss:0, anchor_loss:0
eval -- loss:35.476, x_loss:1.01912, z_loss:1.01747, alt_x_loss:34.4586,
anchor_loss:0.0529254
epoch: 40
train -- loss:2.03436, x_loss:0.735872, z_loss:0.91446, alt_x_loss:0.384031,
x_constraint_loss:0, anchor_loss:0
eval -- loss:27.3714, x_loss:0.890227, z_loss:0.886908, alt_x_loss:26.4845,
anchor_loss:0.0707749
epoch: 49
train -- loss:1.74971, x_loss:0.689517, z_loss:0.824003, alt_x_loss:0.236194,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.8241, x_loss:0.852954, z_loss:0.848932, alt_x_loss:25.9752,
anchor_loss:0.0762041

```

```

=====
Starting training stage: stage3_decoder_response

```

```

Mode: standard
=====

```

```

Train encoder: True
Train decoder: True
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 50, 'lr':

```

```

0.0001, 'beta': 0.5, 'lamb': 0.8, 'tau': 3.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
0.0, 'num_slices': 500}
epoch: 0
train -- loss:2.15318, x_loss:0.733684, z_loss:0.766999, alt_x_loss:0.390913,
x_constraint_loss:0, anchor_loss:0
eval -- loss:27.4224, x_loss:0.897241, z_loss:0.857816, alt_x_loss:26.5646,
anchor_loss:0.0756586
epoch: 20
train -- loss:1.59351, x_loss:0.791638, z_loss:0.821966, alt_x_loss:0.180038,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.6525, x_loss:0.967401, z_loss:0.811138, alt_x_loss:25.8413,
anchor_loss:0.0831531
epoch: 40
train -- loss:1.98315, x_loss:0.773817, z_loss:0.863122, alt_x_loss:0.301913,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.7667, x_loss:0.944671, z_loss:0.782545, alt_x_loss:25.9842,
anchor_loss:0.0864714
epoch: 49
train -- loss:1.57121, x_loss:0.765562, z_loss:0.761153, alt_x_loss:0.193168,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.0119, x_loss:0.950439, z_loss:0.77631, alt_x_loss:25.2356,
anchor_loss:0.0871485

```

```

=====
Starting training stage: stage4_encoder_distribution_polish

```

```

Mode: standard

```

```

=====
Train encoder: True
Train decoder: False
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 50, 'lr': 3e-05,
'beta': 1.0, 'lamb': 4.0, 'tau': 0.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d': 0.0,
'num_slices': 500}
epoch: 0
train -- loss:3.83055, x_loss:0.840847, z_loss:0.747426, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.4391, x_loss:0.985472, z_loss:0.755031, alt_x_loss:25.6841,
anchor_loss:0.0872296
epoch: 10
train -- loss:3.46301, x_loss:0.78537, z_loss:0.669411, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.369, x_loss:0.989255, z_loss:0.745868, alt_x_loss:25.6231,
anchor_loss:0.0873756
epoch: 20
train -- loss:3.88676, x_loss:0.868698, z_loss:0.754514, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.0695, x_loss:0.986803, z_loss:0.73706, alt_x_loss:25.3325,
anchor_loss:0.0872547
epoch: 30

```

```

train -- loss:3.59097, x_loss:0.869034, z_loss:0.680485, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.4529, x_loss:0.982004, z_loss:0.746628, alt_x_loss:25.7063,
anchor_loss:0.0872662
epoch: 40
train -- loss:4.08, x_loss:0.828954, z_loss:0.812761, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.0701, x_loss:0.982363, z_loss:0.740278, alt_x_loss:25.3298,
anchor_loss:0.0872591
epoch: 49
train -- loss:3.60431, x_loss:0.812819, z_loss:0.697872, alt_x_loss:0,
x_constraint_loss:0, anchor_loss:0
eval -- loss:25.7791, x_loss:0.979833, z_loss:0.741138, alt_x_loss:25.038,
anchor_loss:0.0871848

```

```

=====
Starting training stage: stage5_z_cycle_inverse_polish

```

```

Mode: z_cycle
=====

```

```

Custom z-cycle stage

```

```

Train encoder: True

```

```

Train decoder: False

```

```

z_cycle_weight: 8.0

```

```

transverse_weight: 4.0

```

```

epoch 0271 (local 0001) | train_total=4.5317e+00 | train_x=4.2207e-02 |
train_zprior=6.8945e-01 | train_zcycle=3.6594e-01 | train_trans=4.5756e-02 |
eval_total=4.1279e+00

```

```

epoch 0280 (local 0010) | train_total=2.9889e+00 | train_x=2.5222e-01 |
train_zprior=3.6581e-01 | train_zcycle=2.3092e-01 | train_trans=3.9425e-02 |
eval_total=2.7337e+00

```

```

epoch 0290 (local 0020) | train_total=2.1305e+00 | train_x=1.7231e-01 |
train_zprior=2.6469e-01 | train_zcycle=1.5951e-01 | train_trans=3.8192e-02 |
eval_total=1.9089e+00

```

```

epoch 0300 (local 0030) | train_total=1.8456e+00 | train_x=1.5101e-01 |
train_zprior=2.2455e-01 | train_zcycle=1.3792e-01 | train_trans=3.5518e-02 |
eval_total=1.6651e+00

```

```

epoch 0310 (local 0040) | train_total=1.7156e+00 | train_x=1.4190e-01 |
train_zprior=2.0735e-01 | train_zcycle=1.2759e-01 | train_trans=3.4575e-02 |
eval_total=1.5407e+00

```

```

epoch 0320 (local 0050) | train_total=1.6169e+00 | train_x=1.3668e-01 |
train_zprior=1.9155e-01 | train_zcycle=1.2090e-01 | train_trans=3.2488e-02 |
eval_total=1.4506e+00

```

```

Skipping disabled stage: stage6_optional_balanced_finetune

```

```

Training complete.

```

```

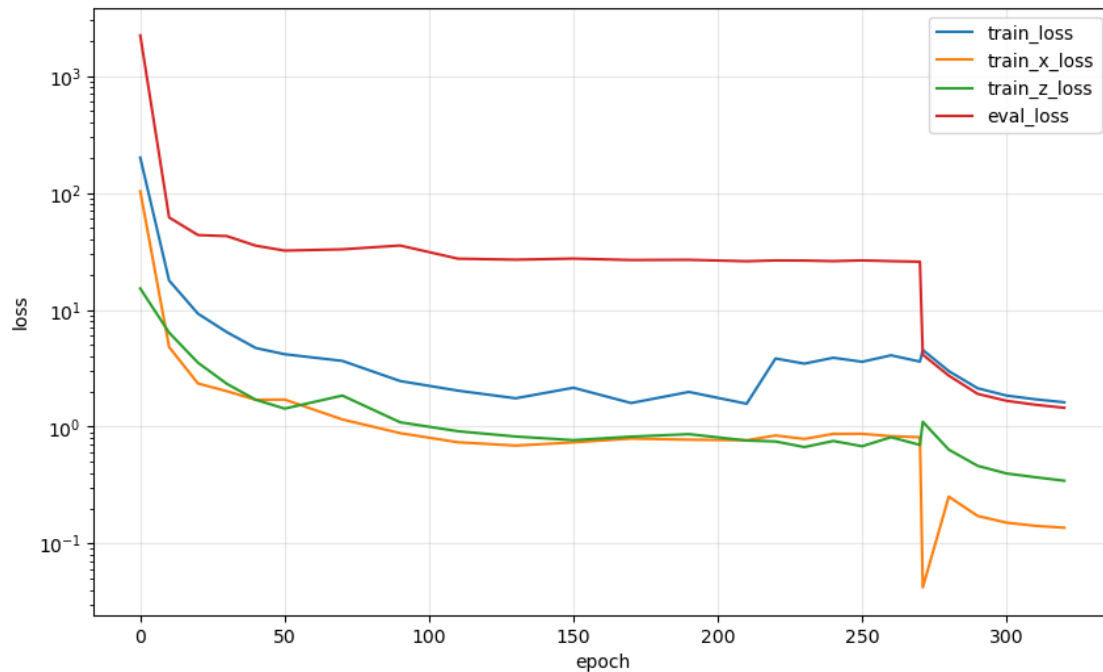
Final eval losses: {'loss': 1.450614720582962, 'x_reco': 0.13482974842190742,
'z_prior': 0.15546858124434948, 'z_cycle': 0.11599460430443287, 'transverse':
0.0192227428779006}

```

CPU times: total: 36min 55s
Wall time: 37min 8s

9.5 Plot Loss History

```
[49]: def history_to_numpy(values):  
    """Convert history lists containing Python numbers, NumPy values, or Torch_  
→tensors to NumPy."""  
    out = []  
    for v in values:  
        if isinstance(v, torch.Tensor):  
            out.append(v.detach().cpu().item() if v.ndim == 0 else v.detach().  
→cpu().numpy())  
        else:  
            out.append(v)  
    return np.asarray(out, dtype=float)  
  
epoch = history_to_numpy(history['epoch'])  
train_loss = history_to_numpy(history['train_loss'])  
train_x_loss = history_to_numpy(history['train_x_loss'])  
train_z_loss = history_to_numpy(history['train_z_loss'])  
eval_loss = history_to_numpy(history['eval_loss'])  
  
fig = plt.figure(figsize=(10, 6))  
  
plt.plot(epoch, train_loss, label='train_loss')  
plt.plot(epoch, train_x_loss, label='train_x_loss')  
plt.plot(epoch, train_z_loss, label='train_z_loss')  
plt.plot(epoch, eval_loss, label='eval_loss')  
  
plt.xlabel('epoch')  
plt.ylabel('loss')  
plt.yscale('log')  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.show()
```



10 Show Results

10.1 Get Results

```
[50]: ## Reset random seeds ##
torch.manual_seed(seed)
np.random.seed(seed)

## Convert model to CPU and evaluation mode ##
model.to('cpu')
model.encoder.output_stats.to('cpu')
model.decoder.output_stats.to('cpu')
model.eval()

all_arrs = {'train': {}, 'val': {}, 'test': {}}
all_arrs['train']['x'] = x_train
all_arrs['train']['z'] = z_train
all_arrs['val']['x'] = x_val
all_arrs['val']['z'] = z_val
all_arrs['test']['x'] = x_test
all_arrs['test']['z'] = z_test

with torch.no_grad():
    for data_key in 'train', 'val', 'test':
        arrs = all_arrs[data_key]
```

```

z_tensor = torch.from_numpy(arrs['z'])
x_tensor = torch.from_numpy(arrs['x'])

arrs['z_decoded'] = model.decode(z_tensor)
arrs['x_encoded'] = model.encode(x_tensor)
arrs['x_reconstructed'] = model.decode(arrs['x_encoded'])

for field, arr in list(arrs.items()):
    if isinstance(arr, torch.Tensor):
        arrs[field] = arr.detach().cpu().numpy()

print('Generated arrays for:', list(all_arrs.keys()))
for key in ['x', 'z', 'x_reconstructed', 'z_decoded']:
    print(key, all_arrs['val'][key].shape)

```

```

Generated arrays for: ['train', 'val', 'test']
x (62394, 8)
z (62394, 8)
x_reconstructed (62394, 8)
z_decoded (62394, 8)

```

11 Save Results

```

[51]: from pathlib import Path

# Save everything under saves/<dataset_name>/
save_dir = Path("saves") / dataset_name
save_dir.mkdir(parents=True, exist_ok=True)

# 1. Save state_dict weights
model_save_path = save_dir / "model_state_dict.pkl"
torch.save(model.state_dict(), model_save_path)
print("Saved model weights to", model_save_path)

# 2. Save full model object, so diagnostic notebooks can directly call model.
    ↪ encode(x)
full_model_save_path = save_dir / "model_full.pt"
torch.save(model, full_model_save_path)
print("Saved full model to", full_model_save_path)

# 3. Save test results
results_save_path = save_dir / "results_test.npz"
np.savez(results_save_path, **all_arrs["test"])
print("Model results saved at", results_save_path)

# 4. Save training history

```

```

history_save_path = save_dir / "history.npz"
history_np = {key: history_to_numpy(value) for key, value in history.items()}
np.savez(history_save_path, **history_np)
print("Training history saved at", history_save_path)

```

Saved model weights to
 saves\cms_doubleelectron_mg5_dyee_dy1j_ptj5_zcycle_v1\model_state_dict.pkl
 Saved full model to
 saves\cms_doubleelectron_mg5_dyee_dy1j_ptj5_zcycle_v1\model_full.pt
 Model results saved at
 saves\cms_doubleelectron_mg5_dyee_dy1j_ptj5_zcycle_v1\results_test.npz
 Training history saved at
 saves\cms_doubleelectron_mg5_dyee_dy1j_ptj5_zcycle_v1\history.npz

12 Validation Plots

```

[52]: from func_utils import Zboson_mass

arrs = all_arrs['val']

m_x = Zboson_mass(arrs['x'])
m_rec = Zboson_mass(arrs['x_reconstructed'])
m_zdec = Zboson_mass(arrs['z_decoded'])

def summarize_mass(name, m):
    bins_summary = np.linspace(70, 110, 81)
    counts, edges = np.histogram(m, bins=bins_summary)
    peak = 0.5 * (edges[np.argmax(counts)] + edges[np.argmax(counts) + 1])
    print(
        f'{name:12s} '
        f'mean={np.mean(m):7.3f}, '
        f'std={np.std(m):7.3f}, '
        f'peak={peak:7.3f}, '
        f'frac_70_110={np.mean((m > 70) & (m < 110)):6.3f}'
    )

summarize_mass('CMS x', m_x)
summarize_mass('x-z-x', m_rec)
summarize_mass('z-x', m_zdec)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

ax.hist(m_x, bins=bins, histtype='step', linewidth=2, label='CMS x')
ax.hist(m_rec, bins=bins, histtype='step', linewidth=2, label='x -> z -> x')
ax.hist(m_zdec, bins=bins, histtype='step', linewidth=2, label='z -> x')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')

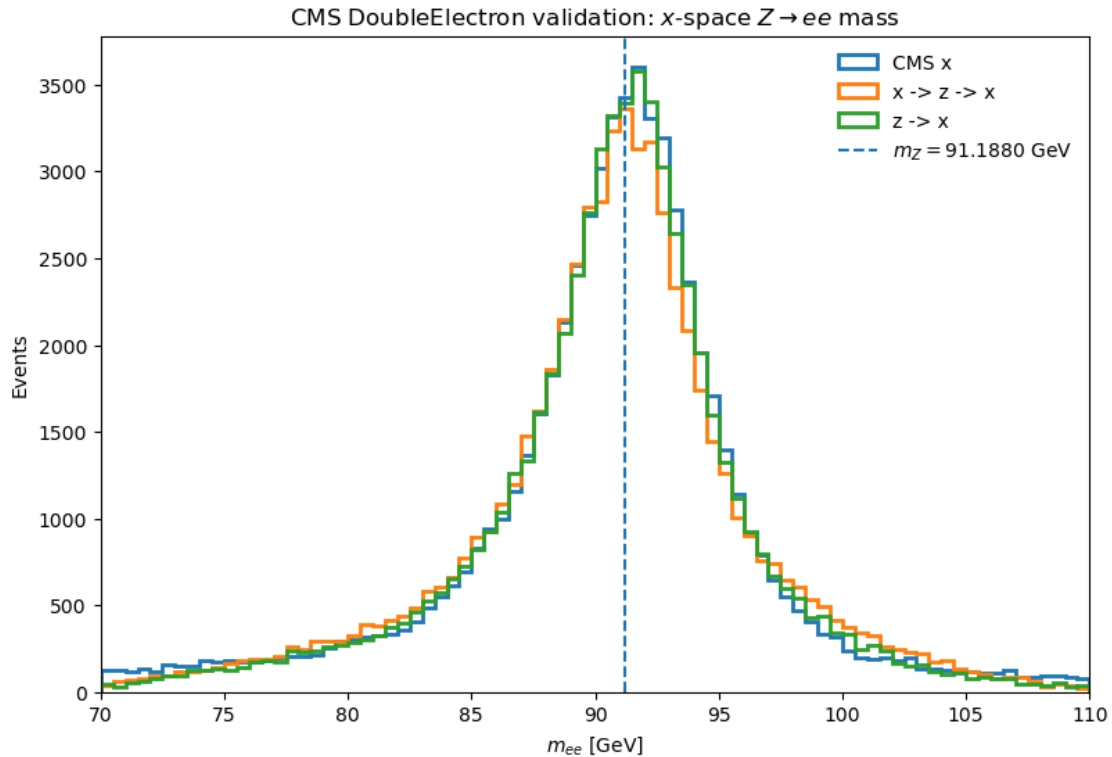
```

```

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Events')
ax.set_xlim(70, 110)
ax.set_title(r'CMS DoubleElectron validation: $x$-space $Z \to ee$ mass')
ax.legend(frameon=False)
plt.show()

```

CMS x	mean= 90.596, std= 5.876, peak= 91.750, frac_70_110= 1.000
x-z-x	mean= 90.522, std= 7.027, peak= 91.250, frac_70_110= 0.985
z-x	mean= 90.593, std= 6.352, peak= 91.750, frac_70_110= 0.989



12.1 Optional generator diagnostic: dilepton transverse momentum

A pure LO MG5 $pp \rightarrow e^+ e^-$ prior has very small dilepton recoil before OTUS. This check helps diagnose whether the generated $z \rightarrow x$ events learn CMS-like transverse recoil.

```

[53]: def dilepton_pt(y):
    px = y[:, 0] + y[:, 4]
    py = y[:, 1] + y[:, 5]
    return np.sqrt(px**2 + py**2)

x_ptll = dilepton_pt(arrs['x'])

```

```

zdec_ptll = dilepton_pt(arrs['z_decoded'])

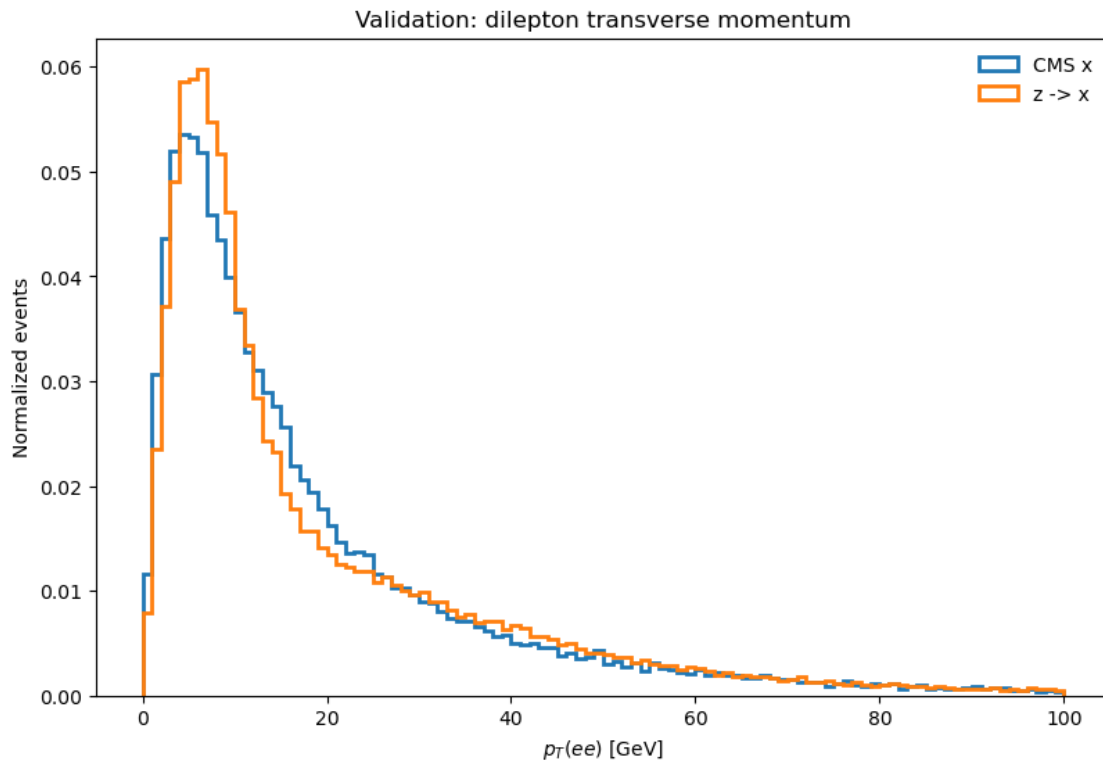
fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(0, 100, 101)

ax.hist(x_ptll, bins=bins, histtype='step', linewidth=2, density=True,
        ↪label='CMS x')
ax.hist(zdec_ptll, bins=bins, histtype='step', linewidth=2, density=True,
        ↪label='z -> x')

ax.set_xlabel(r'$p_T(ee)$ [GeV]')
ax.set_ylabel('Normalized events')
ax.set_title(r'Validation: dilepton transverse momentum')
ax.legend(frameon=False)
plt.show()

print(f'CMS pT(ee) mean: {np.mean(x_ptll):.4f} GeV')
print(f'z -> x pT(ee) mean: {np.mean(zdec_ptll):.4f} GeV')

```



```

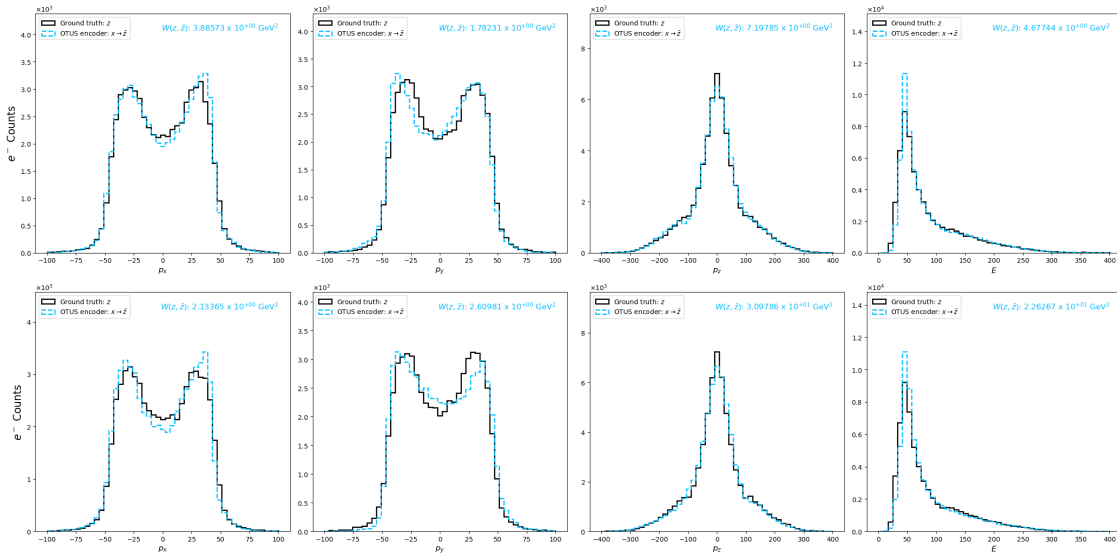
CMS pT(ee) mean: 20.6478 GeV
z -> x pT(ee) mean: 20.9366 GeV

```

```
[54]: from plot_utils import plotFunction

dataList = [arrs['z'], arrs['x_encoded']]
pltDim = (2, 4)
numBins = 50
binsList = [np.linspace(-100., 100., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-400., 400., numBins),
             np.linspace(0., 400., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-400., 400., numBins),
             np.linspace(0., 400., numBins)]
particleNameList = [r'$e^-$'] * 4 + [r'$e^+$'] * 4
nameList = [r'$p_x$', r'$p_y$', r'$p_z$', r'$E$', r'$p_x$', r'$p_y$', r'$p_z$',
            ↪r'$E$']

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```



```
[55]: dataList = [arrs['x'], arrs['x_reconstructed'], arrs['z_decoded']]

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```

